Actividad 2: El Taller del Programador

Configuración del Entorno de Desarrollo y Primer Programa en C++

Autores: Sofia Mojica, Gabriel Zacarías, Mateo Roberts y Leonel De Armas
Institución: Laboratorio de Programación (LPR), Escuela Técnica 5° 3° A-B
Docentes a cargo: Prof. York Mansilla y Prof. Yamil Ganduglia
Fecha: 15 de octubre de 2026

---

1. Entornos de Desarrollo Seleccionados

En la práctica del desarrollo de software moderna, la elección de la herramienta de edición condiciona el flujo de trabajo, el consumo de recursos y la productividad general del equipo. A continuación se detallan las tres plataformas evaluadas durante la práctica:

- **Visual Studio Code (VS Code):** Editor de código fuente ligero, altamente personalizable mediante un ecosistema modular de extensiones. Se ha consolidado como el estándar *de facto* en la industria debido a su flexibilidad, integración nativa con sistemas de control de versiones y bajo consumo de memoria en comparación con entornos integrados completos.
- **Visual Studio Community:** Un entorno de desarrollo integrado (IDE) robusto, de categoría profesional y con un conjunto completo de herramientas integradas. Está orientado a proyectos de gran escala empresarial, desarrollo nativo para plataformas Microsoft y aplicaciones de escritorio complejas que requieren análisis estático avanzado y perfilado de rendimiento.
- **Sublime Text:** Editor de texto sofisticado y ultraligero. Se destaca por su alta velocidad de respuesta, tiempo de carga mínimo y eficiencia operativa en la edición rápida de archivos independientes o scripts individuales sin la sobrecarga de un entorno de proyecto completo.

2. Conceptos Clave del Laboratorio

Para comprender el proceso de transformación de código escrito por un usuario en instrucciones ejecutables por la unidad central de procesamiento (CPU), es imprescindible conceptualizar los elementos fundamentales que intervienen en la cadena de compilación y ejecución:

- **Código Fuente:** Conjunto de instrucciones de programación escritas de forma estructurada en un lenguaje de alto nivel accesible para el ser humano (en este caso, C++). Constituye el insumo base del sistema y requiere traducirse para ser procesado por el hardware.
- **Compilador:** Herramienta de software encargada de traducir de forma integral el código fuente a código máquina binario. Durante este proceso de traducción y enlazado, se genera un archivo ejecutable independiente (con extensión `.exe` en entornos Microsoft Windows).
- **Depurador (Debugger):** Herramienta especializada que ejecuta el programa de manera controlada y secuencial. Permite la inspección en tiempo real de estados de memoria, seguimiento de variables y establecimiento de puntos de interrupción para identificar e aislar fallas lógicas durante la ejecución.
- **Terminal / Consola:** Interfaz de línea de comandos en texto plano utilizada para interactuar directamente con el sistema operativo mediante la emisión de instrucciones directas, ejecución de scripts de compilación y monitoreo del flujo de entrada/salida.
- **Variables de Entorno (PATH):** Registro de configuración del sistema operativo donde se especifican las rutas de acceso a los archivos ejecutables globales. Configurar la variable `PATH` permite invocar herramientas del compilador (como `g++`) desde cualquier directorio en la consola sin especificar su ubicación absoluta.

3. Distribución de Roles y Tareas del Equipo

Con el fin de simular la dinámica de trabajo de un equipo de ingeniería de software, las responsabilidades operativas se estructuraron bajo la siguiente metodología de división del trabajo:

Integrante	Rol Asignado	Responsabilidades Principales
Gabriel Zacarías	Administrador de Entorno	Configuración del entorno base del sistema, instalación del ecosistema MSYS2 y despliegue/verificación del compilador MinGW-w64.
Sofía Mojica	Desarrolladora de Software	Edición y estructuración del código fuente en `src/main.cpp`, integración de la información del equipo y pruebas iniciales de compilación desde la terminal.
Mateo Roberts	Gestor de Evidencias y Git	Recopilación y estandarización visual del proyecto, captura de evidencias del sistema y mantenimiento operativo del archivo `README.md`.
Leonel De Armas	Documentador Técnico	Redacción del glosario técnico, estructuración formal del informe bajo normativa APA 7.ª edición y sistematización en la Carpeta de Campo.

4. Estructura de Carpetas del Proyecto

El proyecto se estructuró siguiendo los estándares de nomenclatura y separación de responsabilidades recomendados para repositorios institucionales y profesionales:/HOLAMUNDO/

├── .gitignore <-- Archivo para omitir carpetas locales (.vscode, binarios)

```
├── README.md              <-- Documentación principal del repositorio y del entorno
├── LICENSE                <-- Términos y condiciones de la licencia del proyecto
├── docs/                  <-- Documentación formal y entregables
│   └── InformeProyectoHOLAMUNDO.pdf <-- ENTREGABLE (Documento en formato APA v7)
├── src/                   <-- Archivos de código fuente
│   └── main.cpp           <-- Programa principal de prueba en C++
└── capturas/              <-- Registro de evidencias de compilación y ejecución
    ├── gpp_version.png    <-- Verificación de versión del compilador g++ en consola
    ├── extensiones_vscode.png <-- Captura del ecosistema de extensiones instaladas
    └── ejecucion_hola_mundo.png <-- Demostración de compilación y ejecución exitosa
```

5. Cuestionario Técnico de Control

1. ¿Qué entorno de desarrollo te parece más intuitivo a primera vista? ¿Por qué?

El entorno más intuitivo evaluado durante el laboratorio es Visual Studio Code. Esta preferencia se debe a su interfaz minimalista y bien estructurada, donde la barra lateral facilita la gestión directa de extensiones y el control de versiones con Git. Asimismo, la posibilidad de integrar la consola de comandos en el mismo panel de trabajo elimina la dispersión de ventanas, reduciendo la carga cognitiva al programar.

2. ¿Qué características de un IDE creés que te facilitan más el aprendizaje?

Las herramientas de asistencia visual y sintáctica son determinantes durante las etapas de formación. En particular, el resaltado de sintaxis mediante código de colores simplifica la detección de errores tipográficos y estructurales. Adicionalmente, el autocompletado inteligente de código y la disponibilidad de una terminal integrada agilizan el ciclo de desarrollo al evitar constantes alternancias entre aplicaciones.

3. De las tareas realizadas, ¿cuál te resultó más difícil de configurar?

El proceso de mayor complejidad técnica fue la configuración y declaración de las Variables de Entorno (PATH) para el compilador MinGW a través de MSYS2. Esta tarea requiere precisión absoluta en la sintaxis de las rutas del sistema; cualquier omisión o la falta de reinicio en las sesiones de la IDE ocasiona la falla de reconocimiento del comando g++ dentro de la terminal.

4. ¿Qué diferencia técnica encontrás entre un "Editor de Texto" (como Sublime) y un "IDE" (como VS Community)?

Un editor de texto funciona primordialmente como una herramienta ligera de modificación de código plano orientada al rendimiento. En contraste, un IDE abarca un ecosistema completo e integrado de forma nativa que incorpora compiladores internos, motores de depuración avanzada (debugging), diseñadores de interfaces gráficas y herramientas de perfilado e integración continua.

5. ¿Qué tipo de aplicaciones sueñan con desarrollar al recibirse cuando lleguen a 6° y 7° año? ¿Por qué?

El equipo contempla dos áreas estratégicas de desarrollo: por un lado, el desarrollo de videojuegos, considerando que C++ es el lenguaje estándar utilizado en motores gráficos de alto rendimiento como Unreal Engine para la optimización de recursos de hardware. Por otro lado, la construcción de aplicaciones web y móviles escalables basadas en arquitecturas modernas (como Flask/Python en el *Backend*) dirigidas a la resolución de problemas logísticos cotidianos.

6. ¿Qué otros lenguajes de programación te gustaría investigar además de C, C++? ¿Por qué?

Resulta de alto interés explorar C# (*C Sharp*), dada su sólida integración con la infraestructura de software de Microsoft y el motor Unity para el desarrollo interactivo. Adicionalmente, Java representa un objetivo clave debido a su amplia adopción corporativa en la arquitectura de sistemas bancarios y el desarrollo nativo para el ecosistema Android.

7. ¿Qué tan importante considerás que es el orden y la nomenclatura de tus archivos y carpetas? ¿Por qué?

El orden y la aplicación rigurosa de estándares de nomenclatura son críticos en el desarrollo profesional. En entornos colaborativos basados en Git, estructurar claramente los módulos (por ejemplo, dividiendo `src/` de `docs/`) y utilizar nombres descriptivos evita la pérdida accidental de datos, simplifica el mantenimiento del código a largo plazo y garantiza un flujo de integración continuo.

8. Si pudieras diseñar el "entorno ideal", ¿qué herramienta o funcionalidad le agregarías para que programar sea más fácil?

Un entorno ideal debería incorporar un sistema de colaboración síncrona en la nube de baja latencia —similar a la edición concurrente de documentos de texto— complementado con una interfaz de voz para la depuración en equipo y un algoritmo automatizado capaz de resolver conflictos complejos en las ramas de Git durante la edición en tiempo real.

9. ¿Cómo creés que la IA puede ayudar a un programador sin que este pierda su capacidad de razonamiento?

La Inteligencia Artificial debe emplearse como un asistente técnico o copiloto, delegando en ella tareas repetitivas de infraestructura como la generación de documentación de soporte, la estructuración de archivos `README` o la identificación de errores sintácticos menores. El desarrollador debe preservar la autoría de la lógica algorítmica fundamental, utilizando la herramienta únicamente para explorar alternativas de optimización.

10. ¿Cuál es tu meta principal para este año en la materia LPR? ¿Por qué?

La meta prioritaria es consolidar la transición conceptual entre la programación interpretada de alto nivel (Python) y la estructuración de bajo nivel orientada a la gestión de memoria en C++. A su vez, se busca asimilar las buenas prácticas de control de versiones y trabajo colaborativo con Git, sentando las bases técnicas requeridas para los proyectos integradores de 6.º y 7.º año y la posterior inserción profesional.

6. Conclusión

La realización de este laboratorio permitió afianzar las bases operativas esenciales para el trabajo en el taller del programador. El análisis del ciclo de vida del software —desde la escritura del fuente hasta su compilación y enlazado— reduce la dependencia de entornos automatizados y otorga al estudiante capacidad diagnóstica sobre el sistema operativo. La compilación exitosa del programa "Hola Mundo" en C++ certifica que el entorno se encuentra correctamente parametrizado para abordar las prácticas algorítmicas avanzadas del ciclo lectivo.

7. Enlaces y Aportes Individuales de Integrantes

1. Leonel De Armas: https://github.com/tomasdearmas99-png/ACTIVIDAD-2-EL-TALLER-DEL-PROGRAMADOR---LPR-5-3-A-B.git